

★ PRIMER · UNIT 13

Before the Lecture: Learning by Doing

Unit 13 · Reinforcement Learning

Watch this first if expected value and discounting are hazy

Twenty minutes of groundwork — what a reward is, why a pound tomorrow is worth less than a pound today, and how a table of guesses slowly becomes a strategy.

How to Use This Primer

Two audiences, one deck — read the column that describes you

If this is new to you

- Watch before the lecture.
- Discounting and expected value are here.
- Do slide 8's arithmetic.

If you have met it before

- Skim it — this is revision.
- Slide 7 is what finance changes.
- Slide 16 says if you can skip.

Nothing here is examined on its own — it is the floor the lecture stands on

Where Unit 13 Sits

The course builds one pipeline; here is this unit's stone

Unit 12

done

Machines that read text.

Unit 13

you are here

Learning by trial, reward and error.

Unit 14

next

Cause, explanation and fairness.

Every model so far predicted. This one acts, and lives with the consequences.

What Makes This Different

There is no answer key and no dataset. The agent acts, the world replies, and the reply is the only teacher.

Supervised learning

- A fixed dataset.
- Correct answers given.
- Learn, then deploy.

Reinforcement learning

- No dataset at all.
- Only rewards, after the fact.
- Learning and acting are one.

This is the third paradigm from unit 1's table, finally arriving in full.

The Words the Lecture Assumes

Five terms, and the Q-learning section opens up

Word	What it means	Trade execution
State	the situation right now	shares left, time left
Action	what you may do	sell 100 now, or wait
Reward	the score for that step	price you actually got
Policy	what to do in each state	the execution strategy
Discount	how much the future counts	γ , between 0 and 1

A policy is not a prediction. It is a rule that says what to do, in every situation.

Three Dials, One Table

The lecture turns each in isolation and shows what breaks

Dial	Controls	Too high
γ (gamma)	how far ahead you look	chases distant, uncertain rewards
α (alpha)	how fast beliefs update	forgets everything it knew
ϵ (epsilon)	how often you explore	never exploits what it learned

All three interact. That is why RL is powerful in a simulator and hard in a market.

Why Execution Is the Natural Fit

Predicting price is nearly impossible. Deciding how to spend the next ten minutes selling is tractable.

Prediction

- Signal near the noise floor.
- Everyone is trying.
- Edge decays fast.

Execution

- The order is already decided.
- Only the how remains.
- Costs are measurable.

This is why the first real RL money in finance was made in execution, not in forecasting.

Pricing Patience by Hand

Sell now for a sure 100, or wait for an uncertain outcome

Choice	Outcome	Probability	Contribution
Sell now	100	1.00	100.0
Wait	110	0.50	55.0
Wait	95	0.50	47.5
Wait — total	—	—	102.5

Waiting is worth 102.5 against 100 for certainty. Expected value says wait.

Now Add Discounting

The same choice, but waiting costs you a day

With $\gamma = 0.95$

- Waiting is worth 102.5.
- Discount it: $\times 0.95$.
- That gives 97.4.

The verdict flips

- Selling now: 100.
- Waiting, discounted: 97.4.
- Sell now.

Gamma is not a technicality — it is the difference between patience and dithering

Explore Against Exploit

The oldest dilemma in the field, and it has no clean answer

Exploit

- Use the best broker you know.
- Reliable, immediate return.
- You may be missing better.

Explore

- Try an unknown broker.
- Costly on average.
- Sometimes finds something better.

Epsilon-greedy is the simplest answer: explore ϵ of the time, exploit the rest.

How the Q-Table Fills In

Three moves, repeated over thousands of episodes

1 · Act

from the table

Look up the current state and pick the best-scoring action.

2 · Observe

the reward

The world replies with a payoff and a new state.

3 · Update

the estimate

Nudge the old guess towards what actually happened.

Every cell starts at zero and is wrong. Repetition, not cleverness, is what fixes them.

Q-Learning and Its Bigger Sibling

The same idea, once the state space stops fitting in a table

Method	Stores values in	Suits
Q-learning	a table, state by action	small, discrete problems
DQN	a neural network	large or continuous states
LOXM-style	a network, live in a market	real execution desks

The network replaces the table. Everything about the learning rule stays the same.

Notation You Will Meet

Four symbols carry the whole unit

Symbol	Read it as	It stands for
$Q(s,a)$	Q of s a	how good action a is in state s
γ	gamma	the discount on future reward
α	alpha	how fast you update a belief
ϵ	epsilon	how often you explore instead of exploit

Q is just a table of guesses. Learning is the process of making those guesses less wrong.

The Python You Will See

You do not need to write it yet — you need to recognise it

Line	What it does
<code>Q = np.zeros((n_states, n_actions))</code>	the table, all guesses at zero
<code>np.argmax(Q[s])</code>	the best action you currently know
<code>if random() < eps: a = random_action()</code>	the exploration branch
<code>Q[s,a] += alpha * (r + gamma*Q[s2].max() - Q[s,a])</code>	the whole learning rule
<code>env.step(a)</code>	act, and receive reward and next state

That fourth line is all of Q-learning. Everything else is bookkeeping around it.

Three Things Beginners Get Wrong

Each one costs a week if you carry it into unit 14

1

"RL will learn to predict the market"

It learns to act. Pointed at price prediction it inherits every problem units 1 to 10 described.

2

"A good simulator is a detail"

The agent learns the simulator, not the market. If your simulator has no market impact, it will learn to ignore it.

3

"Exploration is wasted money"

Without it the agent locks onto the first thing that worked and never discovers anything better.

If one of these surprised you, that is exactly what the primer was for

Self-Check Before the Lecture

Answer these three out loud — if you can, you are ready

1	50/50 between 110 and 95. What is the expected value?	102.5
2	Which dial decides how far ahead the agent looks?	gamma
3	What replaces the Q-table when states get large?	a network

Cover the right column, answer, then check. All three come from this deck.

What the Lecture Does With This

You bring the vocabulary; the lecture brings the argument

1

It sets up the loop

State, action, reward and policy, with patience priced by hand.

2

It fills a Q-table on stage

Three episodes, worked through cell by cell, then the three dials turned one at a time.

3

It ends in code

The problem set runs epsilon-greedy on two brokers, then fills a Q-table by hand.

Next: Unit 13 lecture — Learning to Act